Smart Toll AI Gate

Interview Preparation Guide

Comprehensive Version with Detailed Explanations

1. Project Overview

Smart Toll AI Gate is a full-stack web application that automates toll booth management using AI. The operator captures a vehicle image using a webcam. Google Gemini AI analyzes the image, identifies the vehicle type and license plate, and calculates the toll fee automatically. All transactions are stored in Firebase Firestore and dashboards update in real-time.

2. Problem & Solution

Problems:

• Manual toll checking causes delays and long queues.

• Human errors may lead to wrong fee calculation.

• Traditional systems lack centralized real-time monitoring.

Solutions:

• AI automatically detects vehicle details.

• Fee calculation happens instantly.

• All toll records sync in real-time across dashboards.

3. Tech Stack

Technology	Purpose
React + TypeScript	Frontend UI with type safety
Node.js + Express	Backend API & security proxy
Firebase Firestore	Real-time cloud database
Firebase Auth	User authentication
Google Gemini AI	Vehicle image analysis
Tailwind CSS	UI styling

4. Key Concepts

• **Backend Proxy:** Express server hides the Gemini API key from the browser.

• **Real-Time Updates:** Firestore onSnapshot() pushes updates instantly.

• **TypeScript:** Adds type safety and reduces runtime bugs.

• **Authentication:** Firebase Auth manages secure login.

5. Project Flow

Operator logs in → Opens webcam → Captures vehicle photo → React sends image to Express server → Express calls Gemini AI → Gemini returns vehicle type, plate number, and toll fee → Firestore stores transaction → All dashboards update instantly.

6. Interview Questions & Detailed Answers

Q: Tell me about your project.

I built Smart Toll AI Gate, an AI-powered toll management system that automatically detects vehicles and calculates toll fees using Google Gemini AI. The project solves real-world problems like manual toll checking delays and human errors.

Q: Why React?

I used React because it helps build fast and interactive user interfaces. It also supports reusable components, which makes development easier and organized. This allows for efficient state management and faster rendering of updates.

Q: Why Express?

I used Express.js for backend API handling and secure communication with Gemini AI. It also helps protect the API key from direct frontend exposure by acting as a security proxy between the client and external services.

Q: What is Firebase?

Firebase is a cloud-based platform provided by Google. I used Firebase Firestore as a real-time database to store and sync toll transaction data instantly, and Firebase Auth for user authentication and security management.

Q: Difference between SQL & Firestore?

SQL stores data in tables and rows with predefined schemas. Firestore stores data in collections and documents with flexible structure. Firestore supports real-time updates easily, while SQL databases usually need manual refresh or server-side handling for updates.

Q: Why did you use Firebase?

Firestore provides real-time updates, so all dashboards sync instantly without page refresh. This is critical for a toll management system where multiple operators need live transaction data.

Q: Why AI better than OCR?

OCR mainly reads text and number plates. But AI models like Gemini Vision AI can analyze complete vehicle details from images, including type, color, and category. This provides more comprehensive and accurate toll calculations.

Q: Why TypeScript instead of JavaScript?

TypeScript catches errors during development and improves code reliability. It adds type safety to the codebase, reducing runtime bugs and making the code more maintainable.

Q: Why did you create a backend server?

To protect the Gemini API key. The browser never sees the secret key. The backend acts as a secure proxy, preventing API key exposure in client-side source code.

Q: How does the backend protect the API key?

Instead of calling Gemini API directly from the frontend, I used an Express backend as a secure proxy. This prevents exposing the API key in browser source code and adds an extra layer of security.

Q: What was the biggest challenge?

One major challenge was securing the API and handling real-time data updates properly. I also faced issues while integrating AI response with frontend flow. The solution involved implementing proper error handling

and optimizing Firestore queries for real-time sync.

7. Quick Revision Checklist

- ✓ Backend Proxy protects API keys from browser exposure
- ✓ onSnapshot() enables real-time updates in Firestore
- ✓ Gemini AI analyzes vehicle images for type, plate, and category
- ✓ Firebase Firestore handles authentication and cloud storage
- ✓ TypeScript improves code quality and catches errors early
- ✓ React components are reusable and make UI development efficient
- ✓ Express server acts as secure intermediary between frontend and external APIs

Final Interview Tip

Speak clearly and explain the project naturally. Interviewers mainly check whether you understand your own project. Be ready to explain your design choices, challenges faced, and how you solved them. Demonstrate enthusiasm and technical clarity.